CHAPTER 3

Database Architectures and the Web

Chapter Objectives

In this chapter you will learn:

- The meaning of the client–server architecture and the advantages of this type of architecture for a DBMS.
- The difference between two-tier, three-tier and *n*-tier client–server architectures.
- The function of an application server.
- The meaning of middleware and the different types of middleware that exist.
- The function and uses of Transaction Processing (TP) Monitors.
- The purpose of a Web service and the technological standards used to develop a Web service.
- The meaning of service-oriented architecture (SOA).
- The difference between distributed DBMSs, and distributed processing.
- The architecture of a data warehouse.
- About cloud computing and cloud databases.
- The software components of a DBMS.
- About Oracle's logical and physical structure.

In Chapter 1, we provided a summary of the historical development of database systems from the 1960s onwards. During this period, although a better understanding of the functionality that users required was gained and new underlying data models were proposed and implemented, the software paradigm used to develop software systems more generally was undergoing significant change. Database system vendors had to recognize these changes and adapt to them to ensure that their systems were current with the latest thinking. In this chapter, we examine the different architectures that have been used and examine emerging developments in Web services and service-oriented architectures (SOA).

Structure of this Chapter In Section 3.1 we examine multi-user DBMS architectures focusing on the two-tier client–server architecture and the three-tier client–server architecture. We also examine the concept of middleware and discuss the different types of middleware that exist in the database field. In Section 3.2 we examine Web services which can be used to provide new types of business services to users, and SOA that promotes the design of loosely coupled and autonomous services that can be combined to provide more flexible composite business processes and applications. In Section 3.3 we briefly describe the architecture for a distributed DBMS that we consider in detail in Chapters 24 and 25, and distinguish it from distributed processing. In Section 3.4 we briefly describe the architecture for a data warehouse and associated tools that we consider in detail in Chapters 31–34. In Section 3.5 we discuss cloud computing and cloud databases. In Section 3.6 we examine an abstract internal architecture for a DBMS and in Section 3.7 we examine the logical and physical architecture of the Oracle DBMS.

3.1 Multi-user DBMS Architectures

In this section we look at the common architectures that are used to implement multi-user database management systems: teleprocessing, file-server, and client-server.

3.1.1 Teleprocessing

The traditional architecture for multi-user systems was teleprocessing, where there is one computer with a single central processing unit (CPU) and a number of terminals, as illustrated in Figure 3.1. All processing is performed within the boundaries of the same physical computer. User terminals are typically "dumb" ones, incapable of functioning on their own, and cabled to the central computer. The terminals send messages via the communications control subsystem of the operating system to the user's application program, which in turn uses the services of

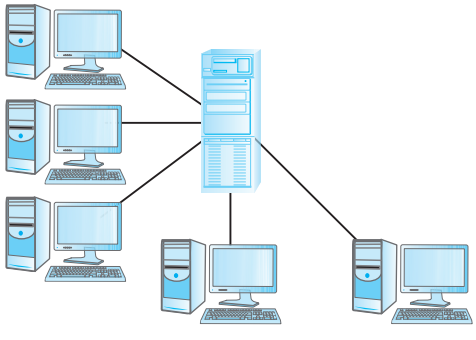


Figure 3.1 Teleprocessing topology.

the DBMS. In the same way, messages are routed back to the user's terminal. Unfortunately, this architecture placed a tremendous burden on the central computer, which had to not only run the application programs and the DBMS, but also carry out a significant amount of work on behalf of the terminals (such as formatting data for display on the screen).

In recent years, there have been significant advances in the development of high-performance personal computers and networks. There is now an identifiable trend in industry towards **downsizing**, that is, replacing expensive mainframe computers with more cost-effective networks of personal computers that achieve the same, or even better, results. This trend has given rise to the next two architectures: file-server and client-server.

3.1.2 File-Server Architecture

File server A computer attached to a network with the primary purpose of providing shared storage for computer files such as documents, spreadsheets, images, and databases.

In a file-server environment, the processing is distributed about the network, typically a local area network (LAN). The file-server holds the files required by the applications and the DBMS. However, the applications and the DBMS run on each workstation, requesting files from the file-server when necessary, as illustrated in Figure 3.2. In this way, the file-server acts simply as a shared hard disk drive. The DBMS on each workstation sends requests to the file-server for all data that the DBMS requires that is stored on disk. This approach can generate a significant amount of network traffic, which can lead to performance problems. For example, consider a user request that requires the names of staff who work in the branch at 163 Main St. We can express this request in SQL (see Chapter 6) as:

```
SELECT fName, lName
FROM Branch b, Staff s
WHERE b.branchNo = s.branchNo AND b.street = '163 Main St';
```

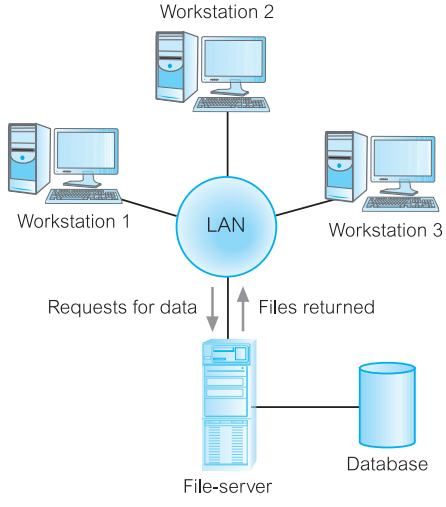


Figure 3.2 File-server architecture.

As the file-server has no knowledge of SQL, the DBMS must request the files corresponding to the Branch and Staff relations from the file-server, rather than just the staff names that satisfy the query.

The file-server architecture, therefore, has three main disadvantages:

(1) There is a large amount of network traffic.

(2) A full copy of the DBMS is required on each workstation.

(3) Concurrency, recovery, and integrity control are more complex, because there can be multiple DBMSs accessing the same files.

3.1.3 Traditional Two-Tier Client–Server Architecture

To overcome the disadvantages of the first two approaches and accommodate an increasingly decentralized business environment, the client–server architecture was developed. Client–server refers to the way in which software components interact to form a system. As the name suggests, there is a **client** process, which requires some resource, and a **server**, which provides the resource. There is no requirement that the client and server must reside on the same machine. In practice, it is quite common to place a server at one site in a LAN and the clients at the other sites. Figure 3.3 illustrates the client–server architecture and Figure 3.4 shows some possible combinations of the client–server topology.

Data-intensive business applications consist of four major components: the database, the transaction logic, the business and data application logic, and the user interface. The traditional two-tier client–server architecture provides a very basic separation of these components. The client (tier 1) is primarily responsible for the *presentation* of data to the user, and the server (tier 2) is primarily responsible for supplying *data services* to the client, as illustrated in Figure 3.5. Presentation services handle user interface actions and the main business and data application logic. Data services provide limited business application logic, typically validation

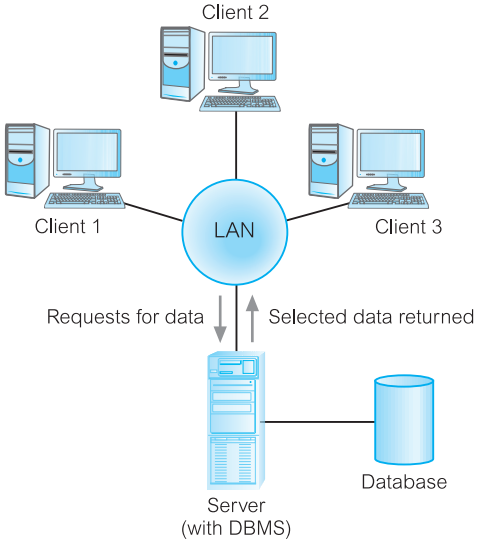


Figure 3.3 Client–server architecture.

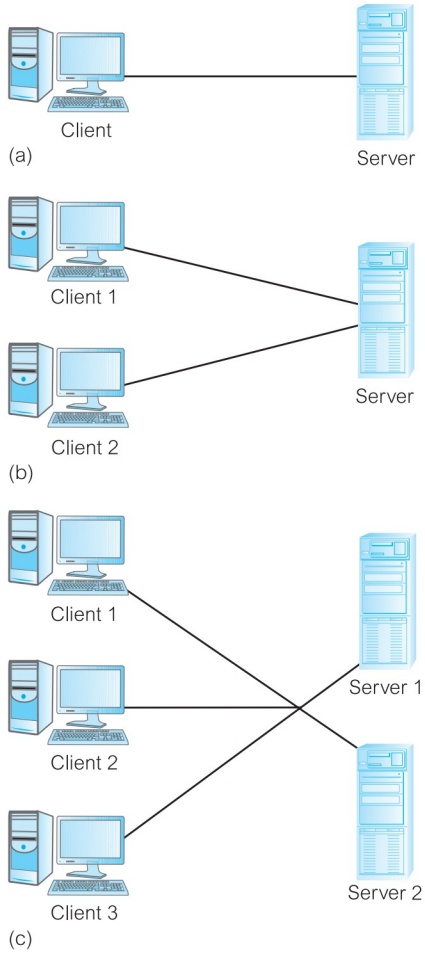


Figure 3.4
Alternative client–server topologies: (a) single client, single server; (b) multiple clients, single server; (c) multiple clients, multiple servers.

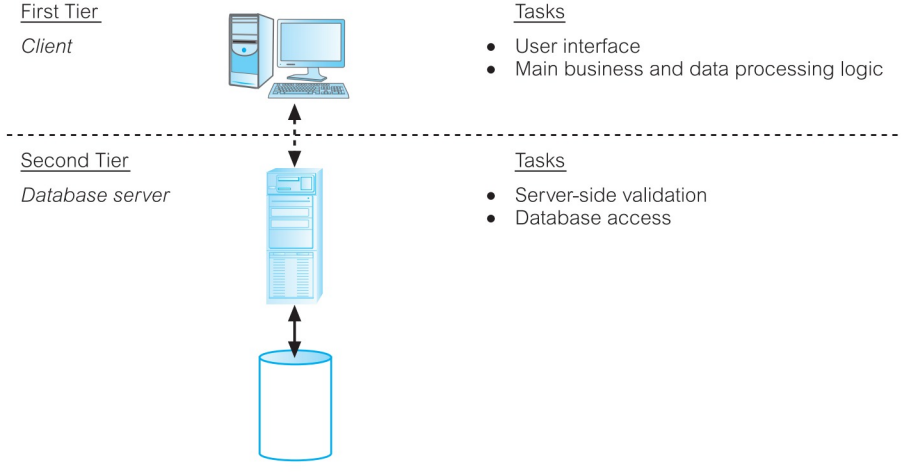


Figure 3.5
The traditional two-tier client–server architecture.

that the client is unable to carry out due to lack of information, and access to the requested data, independent of its location. The data can come from relational DBMSs, object-relational DBMSs, object-oriented DBMSs, legacy DBMSs, or proprietary data access systems. Typically, the client would run on end-user desktops and interact with a centralized database server over a network.

A typical interaction between client and server is as follows. The client takes the user's request, checks the syntax, and generates database requests in SQL or another database language appropriate to the application logic. It then transmits the message to the server, waits for a response, and formats the response for the end-user. The server accepts and processes the database requests, then transmits the results back to the client. The processing involves checking authorization, ensuring integrity, maintaining the system catalog, and performing query and update processing. In addition, it also provides concurrency and recovery control. The operations of client and server are summarized in Table 3.1.

There are many advantages to this type of architecture. For example:

- It enables wider access to existing databases.
- Increased performance: If the clients and server reside on different computers, then different CPUs can be processing applications in parallel. It should also be easier to tune the server machine if its only task is to perform database processing.
- Hardware costs may be reduced: It is only the server that requires storage and processing power sufficient to store and manage the database.
- Communication costs are reduced: Applications carry out part of the operations on the client and send only requests for database access across the network, resulting in less data being sent across the network.
- Increased consistency: The server can handle integrity checks, so that constraints need be defined and validated only in the one place, rather than having each application program perform its own checking.
- It maps on to open systems architecture quite naturally.

Some database vendors have used this architecture to indicate distributed database capability, that is, a collection of multiple, logically interrelated databases distributed over a computer network. However, although the client–server architecture can be used to provide distributed DBMSs, by itself it does not constitute a distributed DBMS. We discuss distributed DBMSs briefly in Section 3.3 and more fully in Chapters 24 and 25.

TABLE 3.1 Summary of client–server functions.

CLIENT	SERVER
Manages the user interface	Accepts and processes database requests from clients
Accepts and checks syntax of user input	Checks authorization
Processes application logic	Ensures integrity constraints not violated
Generates database requests and transmits to server	Performs query/update processing and transmits response to client
Passes response back to user	Maintains system catalog Provides concurrent database access Provides recovery control

3.1.4 Three-Tier Client–Server Architecture

The need for enterprise scalability challenged the traditional two-tier client–server model. In the mid 1990s, as applications became more complex and could potentially be deployed to hundreds or thousands of end-users, the client side presented two problems that prevented true scalability:

- A "fat" client, requiring considerable resources on the client's computer to run effectively. This includes disk space, RAM, and CPU power.
- A significant client-side administration overhead.

By 1995, a new variation of the traditional two-tier client–server model appeared to solve the problem of enterprise scalability. This new architecture proposed three layers, each potentially running on a different platform:

(1) The user interface layer, which runs on the end-user's computer (the *client*).
(2) The business logic and data processing layer. This middle tier runs on a server and is often called the *application server*.
(3) A DBMS, which stores the data required by the middle tier. This tier may run on a separate server called the *database server*.

As illustrated in Figure 3.6, the client is now responsible for only the application's user interface and perhaps performing some simple logic processing, such as input validation, thereby providing a "thin" client. The core business logic of the application now resides in its own layer, physically connected to the client and database server over a LAN or wide area network (WAN). One application server is designed to serve multiple clients.

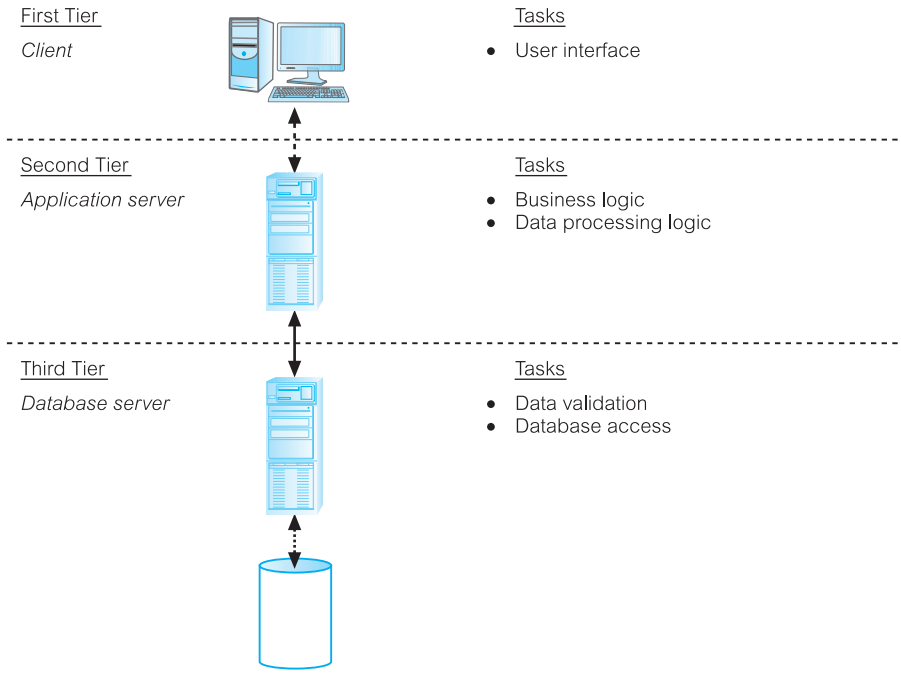


Figure 3.6
The three-tier architecture.

The three-tier design has many advantages over traditional two-tier or single-tier designs, which include:

- The need for less expensive hardware because the client is "thin."
- Application maintenance is centralized with the transfer of the business logic for many end-users into a single application server. This eliminates the concerns of software distribution that are problematic in the traditional two-tier client–server model.
- The added modularity makes it easier to modify or replace one tier without affecting the other tiers.
- Load balancing is easier with the separation of the core business logic from the database functions.

An additional advantage is that the three-tier architecture maps quite naturally to the Web environment, with a Web browser acting as the "thin" client, and a Web server acting as the application server.

3.1.5 *N*-Tier Architectures

The three-tier architecture can be expanded to n tiers, with additional tiers providing more flexibility and scalability. For example, as illustrated in Figure 3.7, the middle tier of the architecture could be split into two, with one tier for the Web

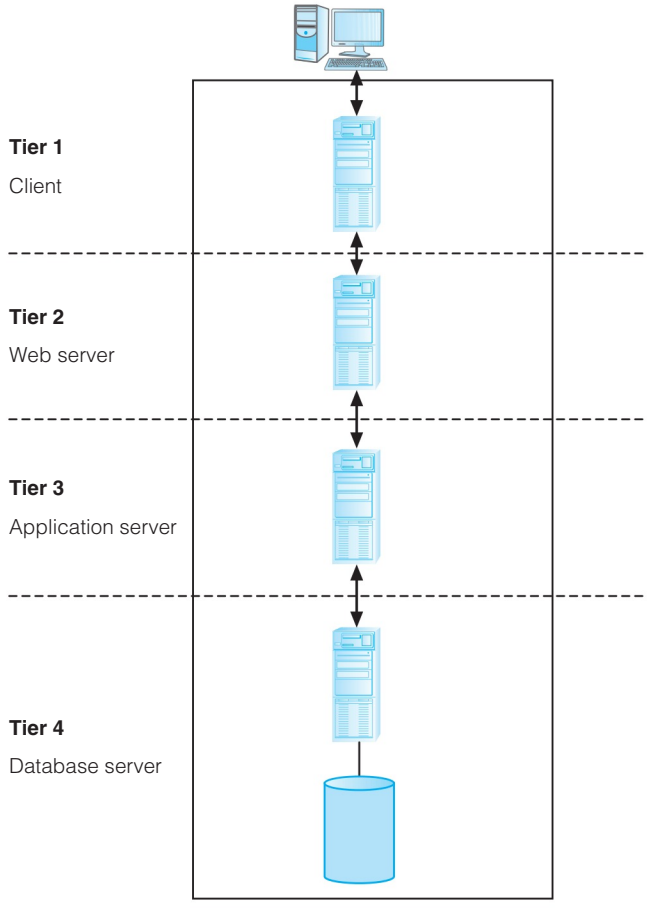


Figure 3.7 Four-tier architecture with the middle tier split into a Web server and application server.

server and another tier for the application server. In environments with a high volume of throughput, the single Web server could be replaced by a set of Web servers (or a *Web farm*) to achieve efficient load balancing.

Application servers

Application server Hosts an application programming interface (API) to expose business logic and business processes for use by other applications.

An application server must handle a number of complex issues:

- concurrency;
- network connection management;
- providing access to all the database servers;
- database connection pooling;
- legacy database support;
- clustering support;
- load balancing;
- failover.

In Chapter 29 we will examine a number of application servers:

- Java Platform, Enterprise Edition (JEE), previously known as J2EE, is a specification for a platform for server programming in the Java programming language. As with other Java Community Process specifications, JEE is also considered informally to be a standard, as providers must agree to certain conformance requirements in order to declare their products to be "JEE-compliant." A JEE application server can handle the transactions, security, scalability, concurrency, and management of the components that are deployed to it, meaning that the developers should be able to concentrate more on the business logic of the components rather than on infrastructure and integration tasks.

  Some well known JEE application servers are WebLogic Server and Oracle GoldFish Server from Oracle Corporation, JBoss from Red Hat, WebSphere Application Server from IBM, and the open source Glassfish Application Server. We discuss the JEE platform and the technologies associated with accessing databases in Section 29.7.
- .NET Framework is Microsoft's offering for supporting the development of the middle tier. We discuss Microsoft .NET in Section 29.8.
- Oracle Application Server provides a set of services for assembling a scalable multitier infrastructure to support e-Business. We discuss the Oracle Application Server in Section 29.9.

3.1.6 Middleware

Middleware Computer software that connects software components or applications.

Middleware is a generic term used to describe software that mediates with other software and allows for communication between disparate applications in a

heterogeneous system. The need for middleware arises when distributed systems become too complex to manage efficiently without a common interface. The need to make heterogeneous systems work efficiently across a network and be flexible enough to incorporate frequent modifications led to the development of middleware, which hides the underlying complexity of distributed systems.

Hurwitz (1998) defines six main types of middleware:

- *Asynchronous Remote Procedure Call (RPC):* An interprocess communication technology that allows a client to request a service in another address space (typically on another computer across a network) without waiting for a response. An RPC is initiated by the client sending a request message to a known remote server in order to execute a specified procedure using supplied parameters. This type of middleware tends to be highly scalable, as very little information about the connection and the session are maintained by either the client or the server. On the other hand, if the connection is broken, the client has to start over again from the beginning, so the protocol has low recoverability. Asynchronous RPC is most appropriate when transaction integrity is not required.
- *Synchronous RPC:* Similar to asynchronous RPC, however, while the server is processing the call, the client is blocked (it has to wait until the server has finished processing before resuming execution). This type of middleware is the least scalable but has the best recoverability.

  There are a number of analogous protocols to RPC, such as:

  – Java's Remote Method Invocation (Java RMI) API provides similar functionality to standard UNIX RPC methods;
  – XML-RPC is an RPC protocol that uses XML to encode its calls and HTTP as a transport mechanism. We will discuss HTTP in Chapter 29 and XML in Chapter 30.
  – Microsoft .NET Remoting offers RPC facilities for distributed systems implemented on the Windows platform. We will discuss the .NET platform in Section 29.8.
  – CORBA provides remote procedure invocation through an intermediate layer called the "Object Request Broker." We will discuss CORBA in Section 28.1.
  – The Thrift protocol and framework for the social networking Web site Facebook.
- *Publish/subscribe:* An asynchronous messaging protocol where *subscribers* subscribe to messages produced by *publishers*. Messages can be categorized into classes and subscribers express interest in one or more classes, and receive only messages that are of interest, without knowledge of what (if any) publishers there are. This decoupling of publishers and subscribers allows for greater scalability and a more dynamic network topology. Examples of publish/subscribe middleware include TIBCO Rendezvous from TIBCO Software Inc. and Ice (Internet Communications Engine) from ZeroC Inc.
- *Message-oriented middleware (MOM):* Software that resides on both the client and server and typically supports asynchronous calls between the client and server applications. Message queues provide temporary storage when the destination application is busy or not connected. There are many MOM products on the market, including WebSphere MQ from IBM, MSMQ (Microsoft Message Queuing), JMS (Java Messaging Service), which is part of JEE and enables the development

of portable, message-based applications in Java, Sun Java System Message Queue (SJSMQ), which implements JMS, and MessageQ from Oracle Corporation.

- *Object-request broker (ORB):* Manages communication and data exchange between objects. ORBs promote interoperability of distributed object systems by allowing developers to build systems by integrating together objects, possibly from different vendors, that communicate with each other via the ORB. The Common Object Requesting Broker Architecture (CORBA) is a standard defined by the Object Management Group (OMG) that enables software components written in multiple computer languages and running on multiple computers to work together. An example of a commercial ORB middleware product is Orbix from Progress Software.
- *SQL-oriented data access:* Connects applications with databases across the network and translates SQL requests into the database's native SQL or other database language. SQL-oriented middleware eliminates the need to code SQL-specific calls for each database and to code the underlying communications. More generally, *database-oriented middleware* connects applications to any type of database (not necessarily a relational DBMS through SQL). Examples include Microsoft's ODBC (Open Database Connectivity) API, which exposes a single interface to facilitate access to a database and then uses drivers to accommodate differences between databases, and the JDBC API, which uses a single set of Java methods to facilitate access to multiple databases. Within this category we would also include *gateways,* which act as mediators in distributed DBMSs to translate one database language or dialect of a language into to another language or dialect (for example, Oracle SQL into IBM's DB2 SQL or Microsoft SQL Server SQL into Object Query Language, or OQL). We will discuss gateways in Chapter 24 and ODBC and JDBC in Chapter 29.

In the next section, we examine one particular type of middleware for transaction processing.

3.1.7 Transaction Processing Monitors

TP Monitor — A program that controls data transfer between clients and servers in order to provide a consistent environment, particularly for online transaction processing (OLTP).

Complex applications are often built on top of several **resource managers** (such as DBMSs, operating systems, user interfaces, and messaging software). A Transaction Processing Monitor, or TP Monitor, is a middleware component that provides access to the services of a number of resource managers and provides a uniform interface for programmers who are developing transactional software. A TP Monitor forms the middle tier of a three-tier architecture, as illustrated in Figure 3.8. TP Monitors provide significant advantages, including:

- *Transaction routing:* The TP Monitor can increase scalability by directing transactions to specific DBMSs.
- *Managing distributed transactions:* The TP Monitor can manage transactions that require access to data held in multiple, possibly heterogeneous, DBMSs. For example, a transaction may require to update data items held in an Oracle DBMS at site 1, an Informix DBMS at site 2, and an IMS DBMS as site 3. TP Monitors normally

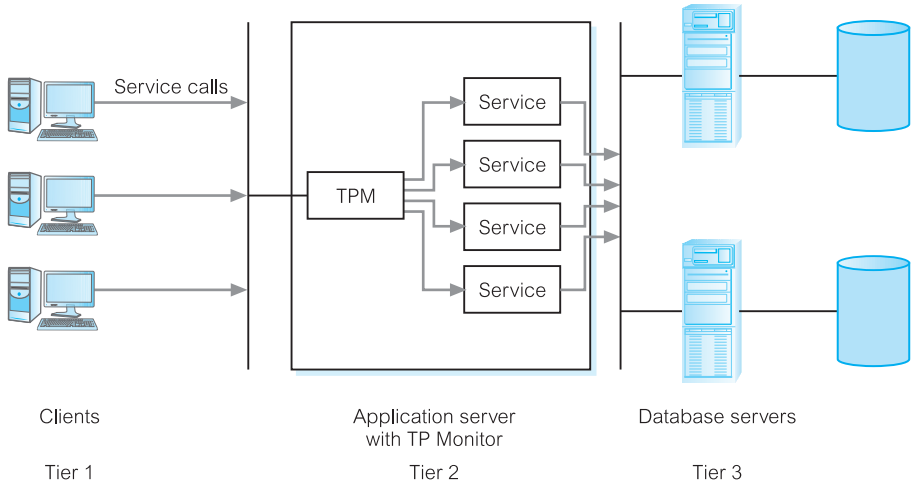


Figure 3.8 The Transaction Processing Monitor as the middle tier of a three-tier client–server architecture.

control transactions using the X/Open Distributed Transaction Processing (DTP) standard. A DBMS that supports this standard can function as a resource manager under the control of a TP Monitor acting as a transaction manager. We discuss distributed transactions and the DTP standard in Chapters 24 and 25.

- *Load balancing:* The TP Monitor can balance client requests across multiple DBMSs on one or more computers by directing client service calls to the least loaded server. In addition, it can dynamically bring in additional DBMSs as required to provide the necessary performance.
- *Funneling:* In environments with a large number of users, it may sometimes be difficult for all users to be logged on simultaneously to the DBMS. In many cases, we would find that users generally do not need continuous access to the DBMS. Instead of each user connecting to the DBMS, the TP Monitor can establish connections with the DBMSs as and when required, and can funnel user requests through these connections. This allows a larger number of users to access the available DBMSs with a potentially much smaller number of connections, which in turn would mean less resource usage.
- *Increased reliability:* The TP Monitor acts as a *transaction manager,* performing the necessary actions to maintain the consistency of the database, with the DBMS acting as a *resource manager*. If the DBMS fails, the TP Monitor may be able to resubmit the transaction to another DBMS or can hold the transaction until the DBMS becomes available again.

TP Monitors are typically used in environments with a very high volume of transactions, where the TP Monitor can be used to offload processes from the DBMS server. Prominent examples of TP Monitors include CICS (which is used primarily on IBM mainframes under z/OS and z/VSE) and Tuxedo from Oracle Corporation. In addition, the Java Transaction API (JTA), one of the Java Enterprise Edition (JEE) APIs, enables distributed transactions to be performed across multiple X/Open XA resources in a Java environment. Open-source implementations of JTA include JBossTS, formerly known as Arjuna Transaction Service, from Red Hat and Bitronix Transaction Manager from Bitronix.

3.2 Web Services and Service-Oriented Architectures

3.2.1 Web Services

Web service A software system designed to support interoperable machine-to-machine interaction over a network.

Although it has been only about 20 years since the conception of the Internet, in this relatively short period of time it has profoundly changed many aspects of society, including business, government, broadcasting, shopping, leisure, communication, education, and training. Though the Internet has allowed companies to provide a wide range of services to users, sometimes called B2C (Business to Consumer), Web services allow applications to integrate with other applications across the Internet and may be a key technology that supports B2B (Business to Business) interaction.

Unlike other Web-based applications, Web services have no user interface and are not aimed at Web browsers. Web services instead share business logic, data, and processes through a programmatic interface across a network. In this way, it is the applications that interface and not the users. Developers can then add the Web service to a Web page (or an executable program) to offer specific functionality to users. Examples of Web services include:

- Microsoft Bing Maps and Google Maps Web services provide access to location-based services, such as maps, driving directions, proximity searches, and geocoding (that is, converting addresses into geographic coordinates) and reverse geocoding.
- Amazon Simple Storage Service (Amazon S3) is a simple Web services interface that can be used to store and retrieve large amounts of data, at any time, from anywhere on the Web. It gives any developer access to the same highly scalable, reliable, fast, inexpensive data storage infrastructure that Amazon uses to run its own global network of Web sites. Charges are based on the "pay-as-you-go" policy, currently $0.125 per GB for the first 50TB/month of storage used.
- Geonames provides a number of location-related Web services; for example, to return a set of Wikipedia entries as XML documents for a given place name or to return the time zone for a given latitude/longitude.
- DOTS Web services from Service Objects Inc., an early adopter of Web services, provide a range of services such as company information, reverse telephone number lookup, email address validation, weather information, IP address-to-location determination.
- Xignite is a B2B Web service that allows companies to incorporate financial information into their applications. Services include US equities information, real-time securities quotes, US equities pricing, and financial news.

Key to the Web services approach is the use of widely accepted technologies and standards, such as:

- XML (extensible Markup Language).
- SOAP (Simple Object Access Protocol) is a communication protocol for exchanging structured information over the Internet and uses a message format based on XML. It is both platform- and language-independent.

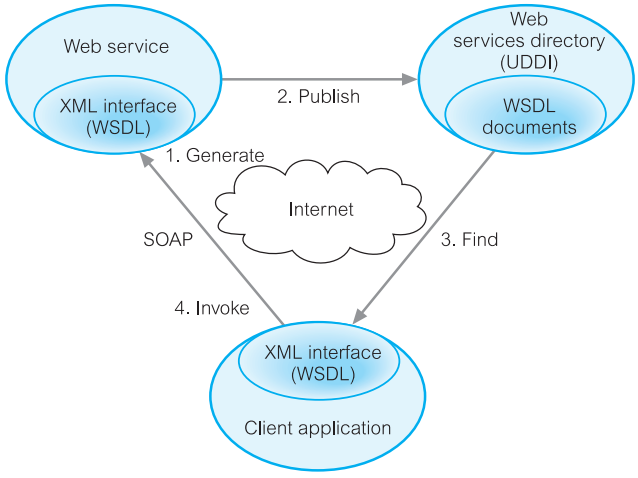


Figure 3.9 Relationship between WSDL, UDDI, and SOAP.

- WSDL (Web Services Description Language) protocol, again based on XML, is used to describe and locate a Web service.
- UDDI (Universal Discovery, Description, and Integration) protocol is a platform-independent, XML-based registry for businesses to list themselves on the Internet. It was designed to be interrogated by SOAP messages and to provide access to WSDL documents describing the protocol bindings and message formats required to interact with the Web services listed in its directory.

Figure 3.9 illustrates the relationship between these technologies. From the database perspective, Web services can be used both from within the database (to invoke an external Web service as a *consumer)* and the Web service itself can access its own database (as a *provider)* to maintain the data required to provide the requested service.

RESTful Web services

Web API is a development in Web services where emphasis has been moving away from SOAP-based services towards Representational State Transfer (REST) based communications. REST services do not require XML, SOAP, WSDL, or UDDI definitions. REST is an architectural style that specifies constraints, such as a uniform interface, that if applied to a Web service creates desirable properties, such as performance, scalability, and modifiability, that enable services to work best on the Web. In the REST architectural style, data and functionality are considered resources and are accessed using Uniform Resource Identifiers (URIs), generally links on the Web. The resources are acted upon by using a set of simple, well-defined HTML operations for create, read, update, and delete: PUT, GET, POST, and DELETE. PUT creates a new resource, which can be then deleted by using DELETE. GET retrieves the current state of a resource in some representation. POST transfers a new state into a resource.

REST adopts a client-server architecture and is designed to use a stateless communication protocol, typically HTTP. In the REST architecture style, clients and servers exchange representations of resources by using a standardized interface and protocol.

We will discuss Web services, HTML, and URIs in Section 29.2.5 and SOAP, WSDL, UDDI in Chapters 29 and 30.

3.2.2 Service-Oriented Architectures (SOA)

SOA

A business-centric software architecture for building applications that implement business processes as sets of services published at a granularity relevant to the service consumer. Services can be invoked, published, and discovered, and are abstracted away from the implementation using a single standards-based form of interface.

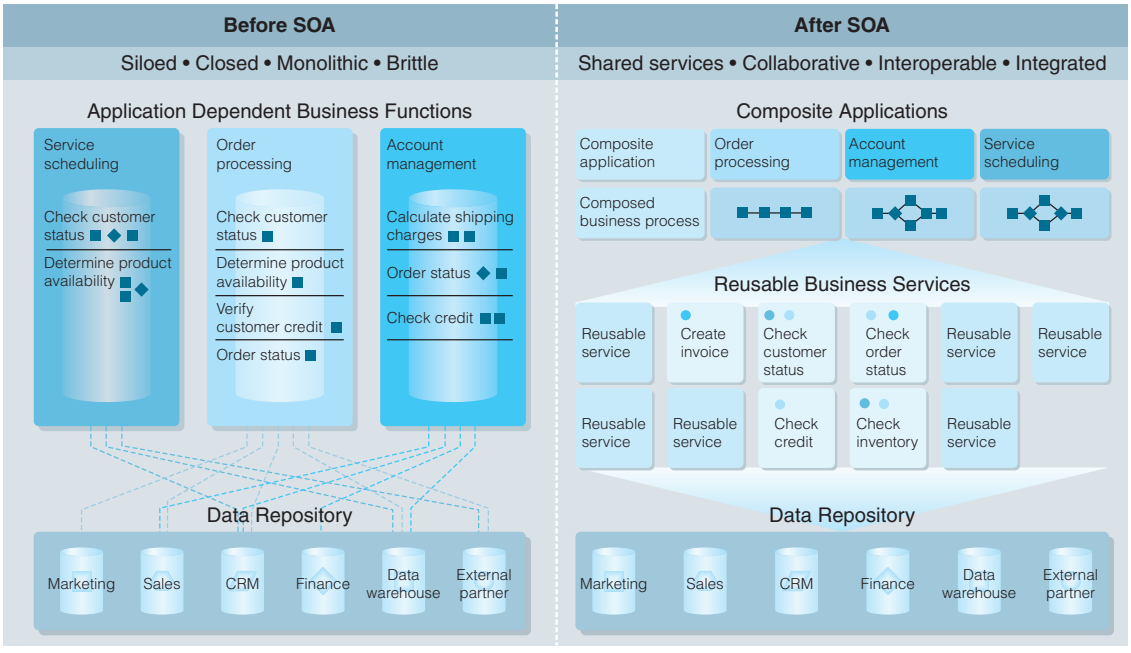
Flexibility is recognized as a key requirement for businesses in a time when IT is providing business opportunities that were never envisaged in the past while at the same time the underlying technologies are rapidly changing. Reusability has often been seen as a major goal of software development and underpins the object-oriented paradigm: object-oriented programming (OOP) may be viewed as a collection of cooperating *objects,* as opposed to a traditional view in which a program may be seen as a group of tasks to compute. In OOP, each object is capable of receiving messages, processing data, and sending messages to other objects. In traditional IT architectures, business process activities, applications, and data tend to be locked in independent, often-incompatible "silos," where users have to navigate separate networks, applications, and databases to conduct the chain of activities that complete a business process. Unfortunately, independent silos absorb an inordinate amount of IT budget and staff time to maintain. This architecture is illustrated in Figure 3.10(a) where we have three processes: Service Scheduling, Order Processing, and Account Management, each accessing a number of databases. Clearly there are common "services" in the activities to be performed by these processes. If the business requirements change or new opportunities present themselves, the lack of independence among these processes may lead to difficulties in quickly adapting these processes.

The SOA approach attempts to overcome this difficulty by designing loosely coupled and autonomous *services* that can be combined to provide flexible composite business processes and applications. Figure 3.10(b) illustrates the same business processes rearchitected to use a service-oriented approach.

The essence of a service, therefore, is that the provision of the service is independent of the application using the service. Service providers can develop specialized services and offer these to a range of service users from different organizations. Applications may be constructed by linking services from various providers using either a standard programming language or a specialized service orchestration language such as BPEL (Business Process Execution Language).

What makes Web services designed for SOA different from other Web services is that they typically follow a number of distinct conventions. The following are a set of common SOA principles that provide a unique design approach for building Web services for SOA:

- *Loose coupling:* Services must be designed to interact on a loosely coupled basis;
- *Reusability:* Logic that can potentially be reused is designed as a separate service;
- *Contract:* Services adhere to a communications contract that defines the information exchange and any additional service description information, specified by one or more service description documents;
- *Abstraction:* Beyond what is described in the service contract, services hide logic from the outside world;



(a) (b)

Figure 3.10 (a) Traditional IT architecture for three business processes; (b) service-oriented architecture that splits the processes into a number of reusable services.

- *Composability:* Services may compose other services, so that logic can be represented at different levels of granularity thereby promoting reusability and the creation of abstraction layers;
- *Autonomy:* Services have control over the logic they encapsulate and are not dependent upon other services to execute this governance;
- *Stateless:* Services should not be required to manage state information, as this can affect their ability to remain loosely-coupled;
- *Discoverability:* Services are designed to be outwardly descriptive so that they can be found and assessed via available discovery mechanisms.

Note that SOA is not restricted to Web services and could be used with other technologies. Further discussion of SOA is beyond the scope of this book and the interested reader is referred to the additional reading listed at the end of the book for this chapter.

3.3 Distributed DBMSs

As discussed in Chapter 1, a major motivation behind the development of database systems is the desire to integrate the operational data of an organization and to provide controlled access to the data. Although we may think that integration and controlled access implies centralization, this is not the intention. In fact, the development of computer networks promotes a decentralized mode of work. This